CENTRO DE ESTUDIOS TECNOLÓGICOS

INDUSTRIAL Y DE SERVICIOS NO. 116

“ANTONIA NAVA DE CATALÁN”

C.C.T.:12DTC0007

PROGRAMACIÓN

MÓDULO 1. Desarrolla software de sistemas informáticos

Submódulo 2. Codifica software de sistemas informáticos

CUADERNILLO DE EJERCICIOS



M.C. Antonio Rodríguez Vázquez

Julio de 2026

Índice

	Pág.
Introducción	1
Datos de la asignatura	2
Unidad 1. Fundamentos del Lenguaje	3
Unidad 2. Arreglos	7
Unidad 3. Clases y Objetos	11
Unidad 4. Métodos	16
Unidad 5. Herencia y Polimorfismo	21
Unidad 6. Archivos	27
Glosario de términos	32
Anexos	35
Bibliografía/Linkografía	37

Introducción

El presente Cuadernillo de Ejercicios pretende ser un complemento de estudio para el Sunmódulo 2. **Codifica software de sistemas informáticos**, del Módulo 1. **Desarrolla software de sistemas informáticos** de la Carrera Técnica en Programación, el cual contiene ejercicios que permitirán a los estudiantes resolver y asimilar mejor los temas propuestos en casa Actividad Clave de la Competencia Laboral de dicho submódulo.

Para realizar los ejercicios propuestos en este cuadernillo será de mayor ayuda que el estudiante esté familiarizado con los conceptos básicos de programación orientada a objetos tales como declaración de variables, estructuras de control selectivas y/o repetitivas, etc.

Se integra al presente cuadernillo un anexo con información útil para el diseño de modelos con UML, se incluye también un glosario de términos que define conceptos importantes en el ámbito de programación.

Por último es importante tener instalado un Lenguaje de Programación Orientada a Objetos en cual se desarrollen estos ejercicios para compilarlos y ejecutarlos, se empleará para el presente cuadernillo el lenguaje de programación C# de Microsoft Visual Studio 2022.

Datos del Submódulo 2 del Módulo 1.

Nombre del Módulo 1 :	Desarrolla software de sistemas informáticos
Nombre del Submódulo 2:	Codifica software de sistemas informáticos
Semestre:	2
Horas teóricas:	24
Horas prácticas:	88
Horas totales:	112
Objetivos Generales del Curso	• Desarrollar software de sistemas informáticos • Diseñar software de sistemas informáticos • Codificar software de sistemas informáticos • Implementar software de sistemas informáticos

Unidad 1. Fundamentos del Lenguaje

Competencias específicas a desarrollar:
• Utilizar técnicas de modelado para la solución de problemas • Aplicar la sintaxis de un lenguaje orientado a objetos. • Aplicar un Lenguaje orientado a objetos para la solución de problemas.
Material y equipo necesario:
• Equipo de cómputo con Lenguaje de Programación Microsoft Visual C# 2022 instalado. • Papel y lápiz

Introducción

En esta primera unidad de la asignatura de programación orientada a objetos se presentan diversos ejercicios que pretenden reforzar el conocimiento adquirido en la primera unidad de dicha asignatura. Se muestran ejercicios de cálculo de operaciones aritméticas básicas y estructuras de control, que muestran los resultados en pantalla, particularmente los ejercicios presentes en este cuadernillo abarcan los siguientes temas;

- Entorno de desarrollo
- Comentarios
- Tipos de datos
- Variables
- Operadores
- Sentencias
- Estructuras de control

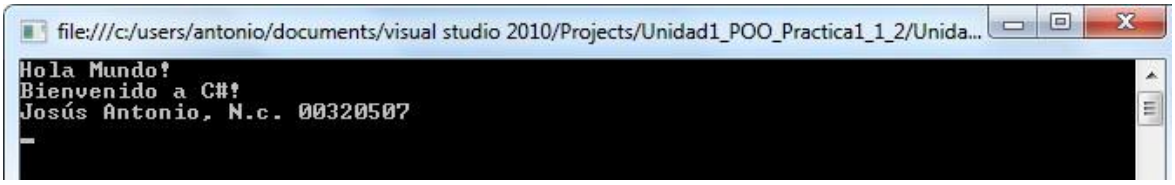
Ejercicio 1.1: Desarrolle un mapa mental de las características principales que contienen los Entorno de Desarrollo Integrado (IDE) de un lenguaje de programación orientado a objetos como C# o Java.

Ejercicio 1.2: Crear una nueva Carpeta en Documentos llamada "*Ejercicios de programación con C#"*, en dicha carpeta se guardarán todos los proyectos que se realizarán en adelante, enseguida; crear un nuevo proyecto de tipo; **Aplicación de Consola** en Microsoft Visual C#, establecer como nombre del proyecto: "*Unidad1_POO_Practica1_1_2"* y guardarlo en la carpeta creada anteriormente.

Dentro de la función *void Main(),* escribir el código necesario que permita:

a) Imprimir en pantalla el texto: " Hola Mundo!"
b) Imprimir en pantalla el texto: " Bienvenido a C#!"
c) Imprimir Nombre del estudiante y su número de control

Al ejecutar el programa, el resultado será la siguiente salida en pantalla.

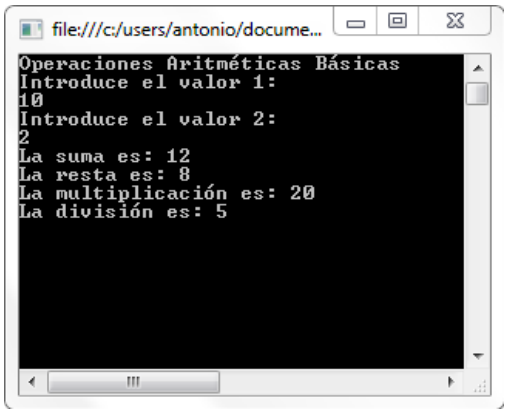


Ejercicio 1.3: De la practica 1.2, identificar en el código las palabras reservadas y realizar la descripción de cada una de estas mediante los símbolos de comentarios (// ó /* */)

Ejercicio 1.4: Crear un programa que permita resolver las 4 operaciones aritméticas básicas (+,-.*, /), para desarrollar esta práctica deberá considerar lo siguiente:

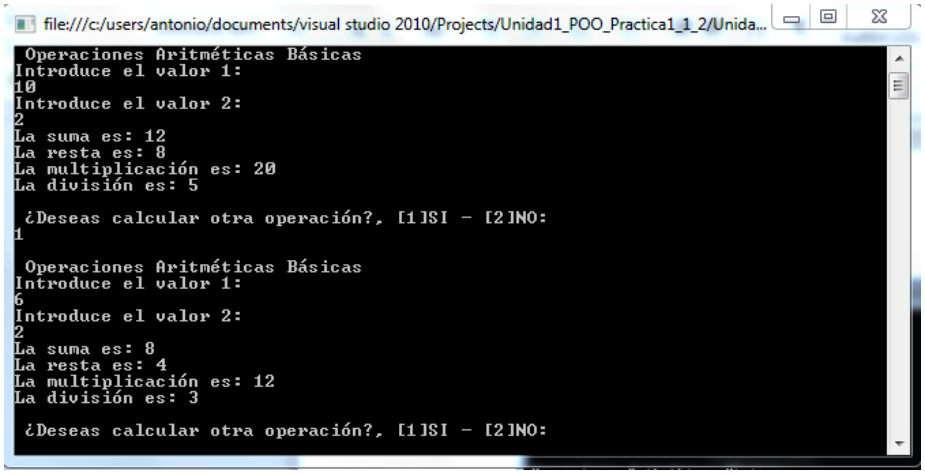
a) La declaración de las variables serán de tipo de datos Double debido a que en la división deben tomarse en cuenta los decimales.
b) Se recibirán 2 valores de entrada, valor1 y valor2, mismos que se utilizarán para realizar las operaciones aritméticas básicas.
c) Presentar los resultados en pantalla mediante una interfaz clara y sencilla.
d) Establecer comentarios al código para entender mejor el programa.

Al ejecutar el programa, el resultado será la siguiente salida en pantalla.



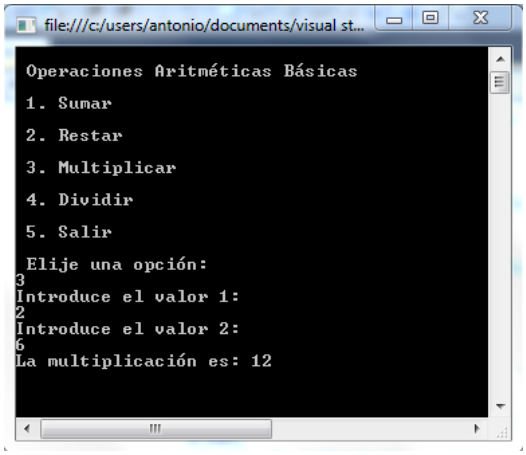
Ejercicio 1.5: De la práctica 1.4, mejorar el código utilizando una estructura de control repetitiva (ciclo do while), esta mejora permitirá realizar una pregunta la cual será; ¿Deseas calcular otra operación?, si la respuesta es 1 entonces el programa pedirá nuevos datos de entrada, si es 2 entonces el programa terminará su ejecución.

Al ejecutar el programa, el resultado será la siguiente salida en pantalla.



Ejercicio 1.6: Realizar la mejora de la Practica 1.5, esta consistirá en aplicar una estructura de control de selección de casos (Switch Case) para que el usuario elija qué operación realizar de las cuatro posibles opciones (casos), cabe mencionar que se mantendrá la opción de calcular otra operación para calcular otras entradas.

Al ejecutar el programa, el resultado será la siguiente salida en pantalla.



Ejercicio 1.7: Realizar un programa que muestre los números enteros del 1 al 100 en pantalla utilizando una estructura de control repetitiva (Ciclo For).

Ejercicio 1.8: Realizar un programa que muestre en pantalla las tablas de multiplicar del 1 al 10 mediante una estructura de control repetitiva anidada (Ciclo For).

Ejercicio 1.9: Realizar la mejora de la practica 1.1.8 introduciendo al programa opción de seleccionar qué tabla de multiplicar calcular y presentar en pantalla, además de que permita calcular otra tabla si así lo desea el usuario

Ejercicio 1.10: Realizar un programa que reciba un número entero de entre el 1 y el 100 el cual representará la edad de una persona con el propósito de calcular en qué categoría se encuentra, las categorías a tomar en cuenta son las siguientes;

a) 1 a 12 años : niño(a)
b) 13 a 17 años: Adolescente
c) 18 a 29 años: Joven
d) 30 a 55 años: Adulto
e) 55 a 70 Adulto mayor
f) 71 a 100 Anciano

En caso de que el número introducido no esté dentro del anterior rango el programa lo marcará con la leyenda, “Edad desconocida” y solicitará otra edad.

Unidad 2. Arreglos

Competencias específicas a desarrollar: • Utilizar técnicas de modelado para la solución de problemas • Aplicar la sintaxis de un lenguaje orientado a objetos. • Aplicar un Lenguaje orientado a objetos para la solución de problemas.
Material y equipo necesario: • Equipo de cómputo con Lenguaje de Programación Microsoft Visual C# 2022 instalado. • Papel y lápiz

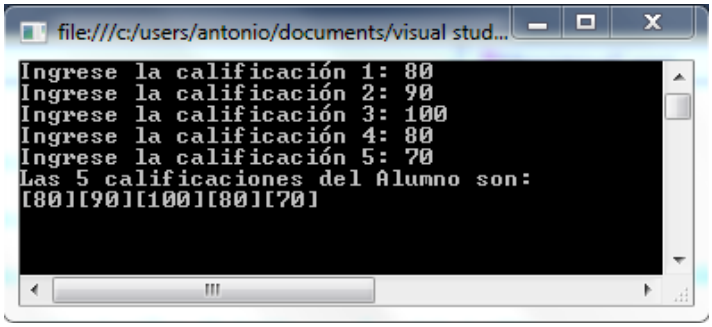
Introducción

En esta unidad se presenta el importante tema de las estructuras de datos: colecciones de elementos de datos relacionados (Arreglos). Los arreglos son estructuras de datos que consisten en elementos de datos del mismo tipo relacionados entre sí.

Un arreglo es una lista de variables del mismo tipo que comparten el nombre, distinguiéndose unas de otras por un índice, los arreglos también son conocidos como vectores para arreglos de una dimensión (Unidimensionales) o matrices para arreglos de dos dimensiones (Multidimensionales).

Ejercicio 2.1 Realizar un programa que permita guardar las calificaciones de 5 unidades de una asignatura impartida en el Tecnológico de San Marcos, este almacenamiento se llevará acabo en un arreglo de tipo vector (Arreglo Unidimensional). Una vez almacenadas todas las calificaciones se mostrarán en pantalla. Será necesario utilizar arreglos y estructuras de control repetitivas.

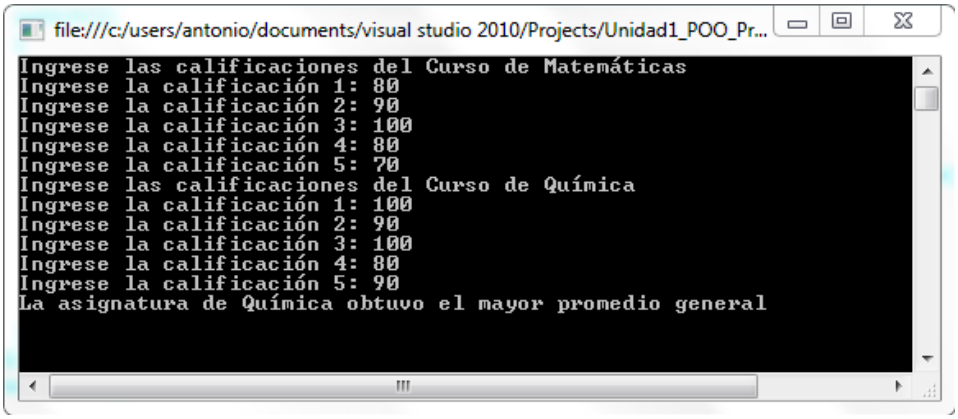
Al ejecutar el programa, el resultado será la siguiente salida en pantalla.



Ejercicio 2.2 Realizar la mejora de la Practica 2.1, el cual consistirá en obtener el promedio general de las calificaciones obtenidas por el estudiante, además se podrá realizar otro ingreso de calificaciones y cálculo de promedio si así ñp lo desea el usuario.

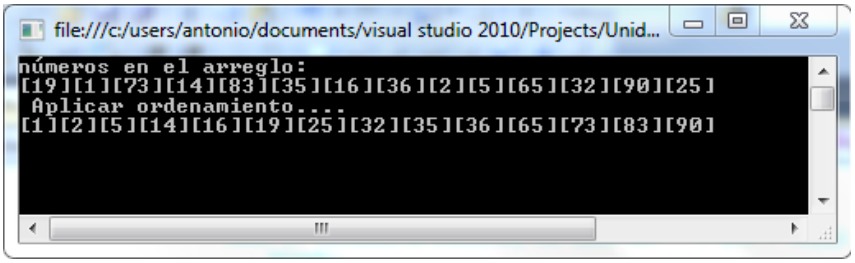
Ejercicio 2.3 Se cuenta con las calificaciones de dos asignaturas (Matemáticas y Química) con el mismo número de unidades (5 unidades por materia), se desea realizar un programa que muestre la asignatura donde se obtuvo el mayor promedio general.

Al ejecutar el programa, el resultado será la siguiente salida en pantalla.



Ejercicio 2.4 Hacer un programa que permita almacenar en un arreglo unidimensional 10 número enteros (del 1 al 100) de forma aleatoria, una vez llenado el arreglo con dichos números, el programa podrá aplicar un método de ordenamiento (puede ser el método de ordenamiento de burbuja) con los números dentro del arreglo. Por último se presentarán estos números de forma ordenada.

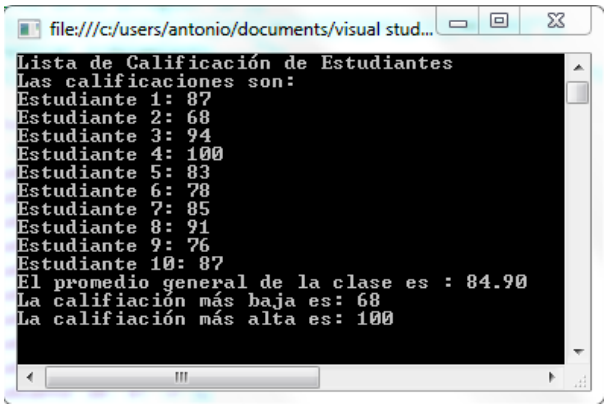
Al ejecutar el programa, el resultado será la siguiente salida en pantalla.



Ejercicio 2.5 Elaborar un programa utilizando Arreglos que permita; introducir calificación a 10 estudiantes (arreglo de 10 posiciones), de estas calificaciones obtener;

a) El promedio general de la clase.
b) La calificación más baja.
c) La calificación más alta.

Al ejecutar el programa, el resultado será la siguiente salida en pantalla.



Ejercicio 2.6 Realiza un programa que al ser ejecutado, presente la siguiente salida en pantalla.

Carrera	Materia1	Materia 2	Materia 3
IGE	Plan de Negocios	Plan de Negocios 2	Costos
IINF	Programación	Programación O.O.	Estructura de Datos
ITIC	F. de Redes	Redes de Computadoras	Redes Emergentes
LI	Desarrollo Web	Negocios Electrónicos	Seguridad Informática

Ejercicio 2.7 Realiza un programa que lea un arreglo bidimensional de 3 X 3 elementos (números enteros positivos menores o iguales a 10), que imprima el arreglo y además que imprima solo los valores de la diagonal.

Ejemplo:

```
1 4 5      5
2 6 2      6
9 5 4      9
```

Ejercicio 2.8 Realiza un programa que sume las siguientes dos matrices e imprimir el resultado en pantalla.

```
1  2 9          8 3 2
2  4 4    +     8 9 4
3  8 1          7 1 9
```

Ejercicio 2.9 Realiza un programa que multiplique las siguientes dos matrices e imprimir el resultado en pantalla.

```
1  2 9          1 3 2
2  4 4    *     3 2 4
               5 1 1
```

Ejercicio 2.10 Elaborar un programa utilizando Arreglos bidimensionales que permita; introducir calificación a 10 estudiantes en 3 materias diferentes, de estas calificaciones obtener;

a) El promedio general de cada estudiante
b) La calificación más baja en la matriz
c) La calificación más alta en la matriz

Al ejecutar el programa, el resultado será la siguiente salida en pantalla.

```
Las calificaciones son:

             Prueba 1 Prueba 2 Prueba 3 Promedio
Estudiante  1       87       96       70    84.33
Estudiante  2       68       87       90    81.67
Estudiante  3       94      100       90    94.67
Estudiante  4      100       81       82    87.67
Estudiante  5       83       65       85    77.67
Estudiante  6       78       87       65    76.67
Estudiante  7       85       75       83    81.00
Estudiante  8       91       94      100    95.00
Estudiante  9       76       72       84    77.33
Estudiante 10       87       93       73    84.33

La calificación más baja en el libro de calificaciones es 65
La calificación más alta en el libro de calificaciones es 100
```

Unidad 3. Clases y Objetos

Competencias específicas a desarrollar: • Utilizar técnicas de modelado para la solución de problemas • Aplicar la sintaxis de un lenguaje orientado a objetos. • Aplicar un Lenguaje orientado a objetos para la solución de problemas.
Material y equipo necesario: • Equipo de cómputo con Lenguaje de Programación Microsoft Visual C# 2022 instalado. • Papel y lápiz

Introducción:

C# es un lenguaje orientado a objetos, lo que significa que todo con lo que vamos a trabajar en este lenguaje son objetos. Un objeto es un agregado de datos y de métodos que permiten manipular dichos datos, y un programa en C# no es más que un conjunto de objetos que interaccionan unos con otros a través de sus métodos.

Una clase es la definición de las características concretas de un determinado tipo de objetos. Es decir, de cuáles son los datos y los métodos de los que van a disponer todos los objetos de ese tipo. Por esta razón, se suele decir que el tipo de dato de un objeto es la clase que define las características del mismo

Ejercicio 3.1 Realizar la investigación sobre los modificadores de acceso en C# y UML. Rellena los espacios en blanco con la respuesta correcta.

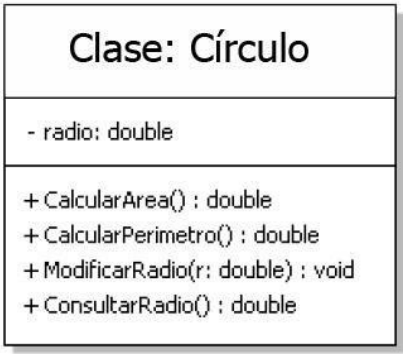
Modificador de Acceso	Codificación en C#	Representación en UML.
Privado		**-**
	public	
Protegido		**#**
Interno	**Internal**	
	Protected internal	**#**

Ejercicio 3.2 Elaborar un programa que defina una clase llamada Automóvil y definir;

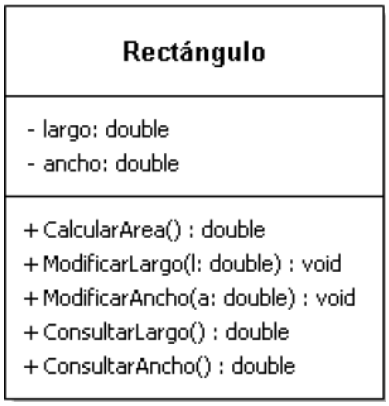
a) Constructor de la clase.
b) 2 atributos privadas.
c) 2 métodos públicos.

Ejercicio 3.3 Realizar una Clase llamada Círculo que tenga como atributo privado el radio de tipo real (Double) y los siguientes métodos:

a) Calcular el área.
b) Calcular el perímetro.
c) Modificar el radio
d) Consultar el valor del radio

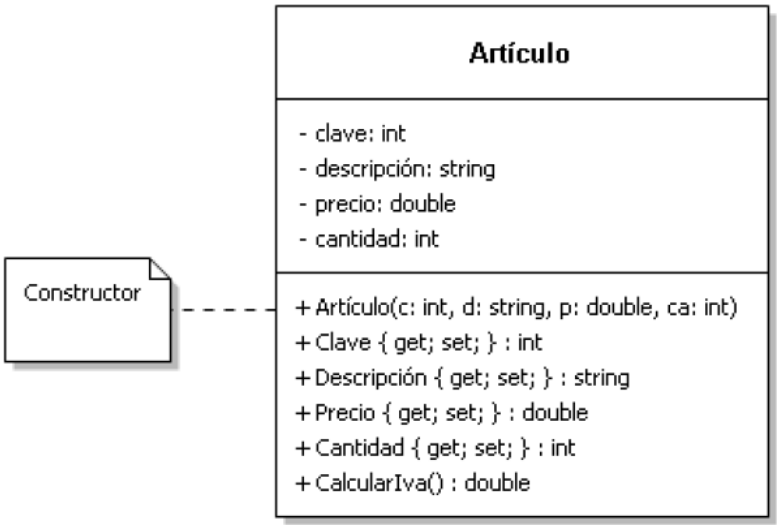


Ejercicio 3.4 Realizar el código para obtener una clase llamada Rectángulo mediante el siguiente diagrama de UML.



Ejercicio 3.5 A partir de la Práctica 3.3., realizar el código que resuelva el siguiente problema: Hay una pared rectangular la cual contiene una ventana rectangular; se requiere un programa que dadas las dimensiones (largo y ancho) de la ventana y de la pared muestre en la pantalla el tiempo necesario para pintar la pared si se sabe que se puede pintar $1m^2$ en 10 minutos.

Ejercicio 3.6 Crear una clase llamada Tienda_comercial cuyo diagrama UML es:

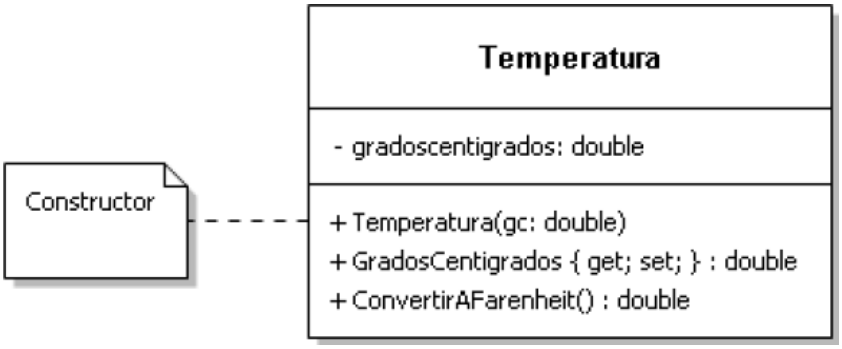


a) Probar esta clase mediante la creación de un objeto que modifique sus valores e imprima el valor del IVA correspondiente.

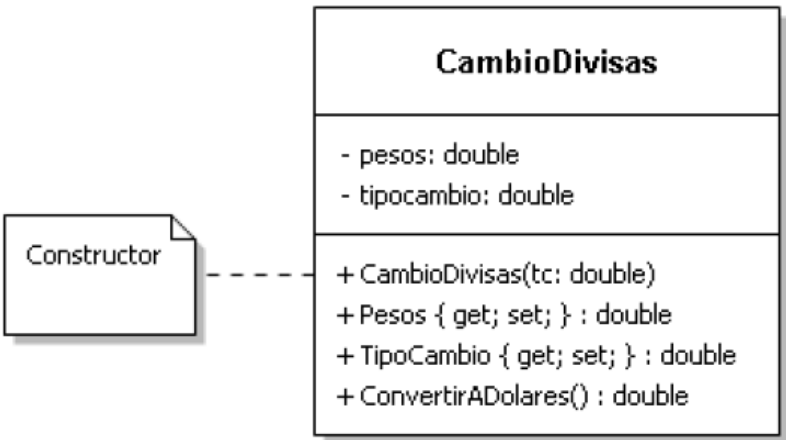
Ejercicio 3.7 Desarrollar una clase que almacene una temperatura dada en grados centígrados y la convierta a grados farenheit, de acuerdo a la fórmula:

$$F = \frac{9}{5}C + 32$$

a) Utilizar el siguiente diagrama de clase diseñado en UML



Ejercicio 3.8 Crear una clase que permita cambiar pesos a dólares, apoyarse con el siguiente diagrama de clase hecho en UML.



Ejercicio 3.9 Realizar la impresión de una llamada telefónica, almacenando la duración de la llamada en minutos y conociendo lo siguiente:

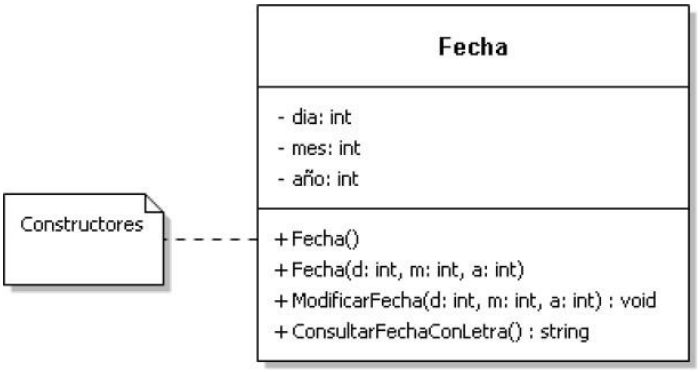
a) Toda la llamada que dure tres minutos o menos tiene un costo de $5
b) Cada minuto adicional cuesta $3

La clase debe tener las siguientes características:

a) Un atributo que almacena la duración de la llamada
b) Una propiedad para validar que la captura sea un número entero positivo.
c) Un método que calcula el costo de la llamada y devuelve el resultado.

Ejercicio 3.10 Crear una clase llamada Fecha, el cual tendrá los siguientes métodos y propiedades;

a) Atributos: Día – Més – Año
b) Constructores:
    1. Para inicializar una fecha específica definida por el programados
    2. Que reciba como parámetro los valores para inicializar la fecha.
c) Métodos públicos:
    1. Para poder modificar el valor de la fecha
    2. Para mostrar en pantalla la fecha usando el formato día/mes/año
    3. Para mostrar en pantalla la fecha poniendo el mes con palabras.
    4. Que permita verificar si una fecha es válida; este método debe ser utilizado por el constructor y el método que va a modificar el valor fijo que se haya definido.
d) Con toda esta información de la clase se deberá crear una aplicación que la pruebe, en este programa se deben crear al menos 2 objetos de tipo *Fecha*, utilizando cada uno de los constructores y después mostrar las fechas correspondientes en la pantalla.



Unidad 4. Métodos

Competencias específicas a desarrollar: • Utilizar técnicas de modelado para la solución de problemas • Aplicar la sintaxis de un lenguaje orientado a objetos. • Aplicar un Lenguaje orientado a objetos para la solución de problemas.
Material y equipo necesario: • Equipo de cómputo con Lenguaje de Programación Microsoft Visual C# 2022 instalado. • Papel y lápiz

Introducción

En esta unidad se presentan ejercicios del uso de métodos en C#. Los métodos son un mecanismo importante para estructurar el código de un programa. Los ejercicios que se presentan en este cuadernillo abarcan los siguientes subtemas de la unidad 4 del programa de estudios de la asignatura de Programación Orientada a Objetos;

- Estructura de un método.
- Cómo se crean y cómo se pueden hacer llamadas a métodos desde una sola clase y desde una clase a otra.
- Ámbito y tiempo de vida de variables.
- Devolver un valor desde un método y a utilizar parámetros para transferir datos a un método o extraerlos de él.
- Sobrecarga de métodos.
- Encapsulamiento.

Un método es una serie de instrucciones C# que han sido agrupadas bajo un nombre determinado. La mayor parte de los lenguajes de programación modernos incluyen un concepto similar; se puede considerar que un método es equivalente a una función, una subrutina, un procedimiento o un subprograma.

Ejercicio 4.1 De acuerdo a la siguiente sintaxis de, Definición de Métodos, definir sus componentes:

```
<tipoRetorno> <nombreMétodo>(<parámetros>)
{
        <cuerpo>
}
```

a) <tipoRetorno>:
b) <nombreMétodo>:
c) <parámetros>:
d) <cuerpo>:

Ejercicio 4.2 De acuerdo a la siguiente sintaxis de llamada a un método, definir sus componentes:

```
<objeto>.<nombreMétodo>(<valoresParámetros>)
```

a) <Objeto>:
b) <NombreMétodo>
c) <ValoresParámtros>

Ejercicio 4.3 Describir cada una de las estructuras de los siguientes métodos.

```
a) public  void suma()
   {
        Cuerpo
   }

b)  public int suma()
    {
        Cuerpo
      Return valor;
    }

c)  public void suma(int n1, int n2)
    {
        Cuerpo
    }

d)  public int (int n1, int n2)
    {
        Cuerpo
     Return valor;
    }
```

Ejercicio 4.4. Realizar una mejora al código presentado en la Práctica 1.6 de la Unidad 1. Establecer 4 métodos para resolver estas operaciones aritméticas básicas, dichos métodos deben recibir 2 parámetros de entrar y devolver un valor a la clase principal para su impresión en pantalla.

Ejercicio 4.5 Describir de forma detallada qué se está definiendo en el siguiente código de una clase y sus métodos.

```
class Sobrecarga
        {
                public void f(int x)
                {}
                public void f(out int x)
                {}
        }
```

Ejercicio 4.6 Describir de forma detallada qué se está definiendo en el siguiente código.

```
class Prueba
    {
        Prueba(int x)
        {
            System.Console.Write("Creado objeto Prueba con x={0}",x);
        }

        public static void Main()
        {
            Prueba p = new Prueba(5);
        }
    }
```

Ejercicio 4.7 Describir de forma detallada qué se está definiendo en el siguiente código.

```
class OverloadingExample
{
        static int Suma(int a, int b)
        {
                return a + b;
        }
        static int Suma(int a, int b, int c)
        {
                return a + b + c;
        }
        static void Main( )
        {
                Console.WriteLine(Suma(1,2) + Suma(1,2,3));
        }
}
```

Ejercicio 4.8 Describir de forma detallada qué operaciones realiza el siguiente código.

```
class MainClass
{
    public static void Main(string[] args)
    {
        int a;
        Console.WriteLine("Ingrese un valor con tres dígitos:");
        a=int.Parse(Console.ReadLine());
        bool digito=verificardigito(a);
        if(digito)
            Console.WriteLine("TIENE 3 DIGITOS");
        else
            Console.WriteLine("TIENE MENOS DE 3 DIGITOS");
        Console.ReadKey();
    }

    public static bool verificardigito(int param)
    {
        if(param>=100)
            return true;
        else
            return false;
    }
}
```

Ejercicio 4.9 Describir de forma detallada qué operaciones realiza en el siguiente código.

```
class MainClass
{
    public static void Main(string[] args)
    {
        int num;
        Console.WriteLine("ingrese un valor:");
        num = int.Parse(Console.ReadLine());
        bool negativo = verificarnegativo(num);
        if (negativo)
            Console.WriteLine("Es un numero negativo");
        else
            Console.WriteLine("El numero no es negativo");
        Console.ReadKey();

    }
    public static bool verificarnegativo(int param)
    {

        if (param < 0)
            return true;
        else
            return false;
    }
}
```

Ejercicio 4.10 Describir de forma detallada qué operaciones realiza en el siguiente código.

```
class Square
{
    public int width, height;

    public int Height
    {
        get { return height; }
        set { height = value; }
    }

    public int Width
    {
        get { return width; }
        set { width = value; }
    }
}
class MainClass
{
    public static void Main()
    {
        Square mySquare = new Square();
        mySquare.Width = 110;
        mySquare.Height = 150;

        Console.WriteLine("width = {0}", mySquare.Width);
        Console.WriteLine("height = {0}", mySquare.Height);
    }
}
```

Unidad 5. Herencia y Polimorfismo

Competencias específicas a desarrollar: • Utilizar técnicas de modelado para la solución de problemas • Aplicar la sintaxis de un lenguaje orientado a objetos. • Aplicar un Lenguaje orientado a objetos para la solución de problemas.
Material y equipo necesario: • Equipo de cómputo con Lenguaje de Programación Microsoft Visual C# 2022 instalado. • Papel y lápiz

Introducción

En esta unidad se manejan dos conceptos muy importantes en lenguajes de programación orientada a objetos;

- La herencia, que es una forma de reutilización de software, en la cual para crear una nueva clase se absorben los miembros de una clase existente y se mejoran con capacidades nuevas o modificadas. Con la herencia, los programadores ahorran tiempo durante el desarrollo de aplicaciones, al reutilizar software de alta calidad, probado y depurado. Esto también incrementa la probabilidad de que el sistema se implemente en forma efectiva.

- El polimorfismo nos permite “programar en forma general”, en vez de “programar en forma específica”. En especial, nos permite escribir aplicaciones que procesen objetos de clases que formen parte de la misma jerarquía de clases, como si todos fueran objetos de la clase.

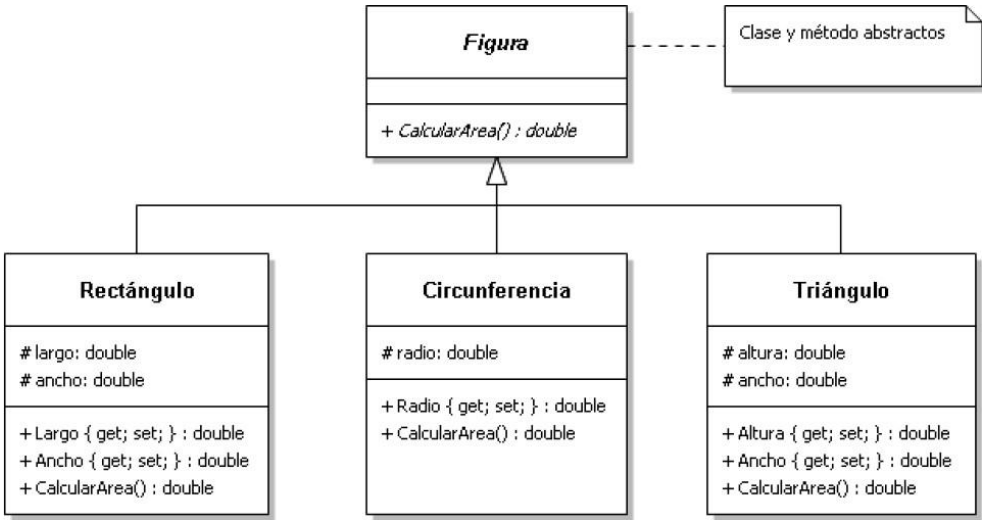
Con el polimorfismo podemos diseñar e implementar sistemas que puedan extenderse con facilidad; pueden agregarse nuevas clases con sólo modificar un poco (o nada) las porciones generales de la aplicación, siempre y cuando las

nuevas clases sean parte de la jerarquía de herencia que la aplicación procesa en forma genérica.

Los ejercicios presentados en este cuadernillo permitirán que los estudiantes elaboraren diagramas de clases en UML y los codifiquen aplicando relaciones de herencia y polimorfismo

Ejercicio 5.1 Diseñar un diagrama de clases mediante una herramienta de programación con UML, que represente una abstracción de las características más importantes de una clase, mediante la Herencia, ejemplo la Clase Ave, hereda características y métodos a una Clase hija llamada Gallina.

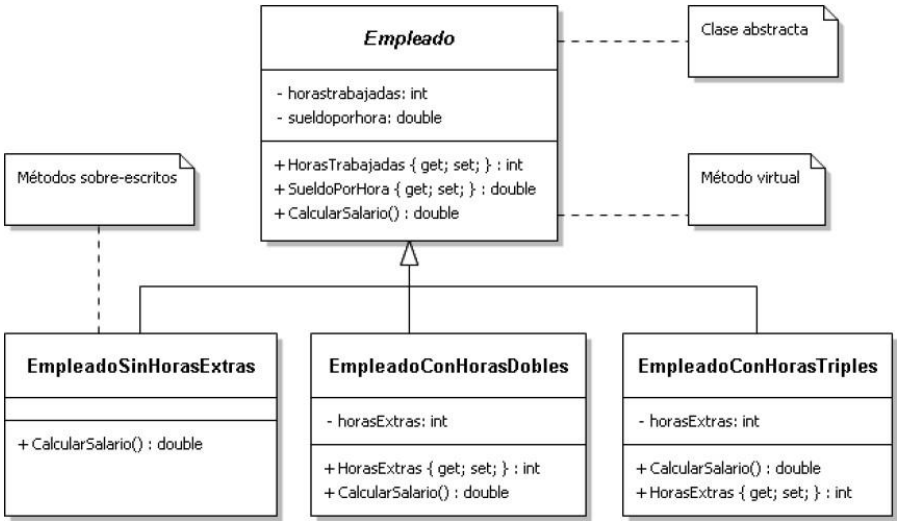
Ejercicio 5.2 Elaborar un programa que permita calcular el área de diversas figuras geométricas utilizando Herencia y Polimorfismo, mediante el siguiente diagrama en UML.



Ejercicio 5.3 Crear un programa que permita a una empresa calcular el salario mensual de sus empleados con base en la cantidad de horas laboradas, el sueldo por hora y teniendo en cuenta los siguientes criterios:

a) Las horas de trabajo normal son 40 horas si se excede se consideran horas extras.
b) Si las horas extras laboradas están entre 41 y 45 cada hora extra se pagará doble.
c) Si las horas laboradas son más de 45, entonces se pagará el triple.

Implemente el programa de acuerdo con el siguiente diagrama de clases UML



Ejercicio 5.4 Diseñar un diagrama UML y codificar su programa para el almacenamiento de libros impresos y audio-libros. Diseñar una clase padre que se llamará *Publicación* que almacene el título y el precio de la publicación, diseñar también dos clases hijas llamadas, *Libro* a la cual se le agregará el número de páginas y *DVD* al cual se le agregará el tiempo de reproducción en minutos. Cada clase tendrá sus propiedades para acceder a sus respectivos datos. Ejemplo de cómo quedará el programa una vez terminado.



Cuando el usuario seleccione el libro impreso, entonces debe activarse el cuadro de texto para capturar el número de páginas y desactivar el cuadro de texto del tiempo en minutos; mientras que si el usuario selecciona el audio-libro, entonces debe activarse el cuadro de texto del tiempo en minutos y desactivar el cuadro de texto del número de páginas

Ejercicio 5.5 Una agencia vendedora de autos desea un sistema computacional para administrar los datos de sus vehículos y clasificarlos por tipo. Todos los autos tienen los siguientes datos:

· Número de serie del motor.
· Marca.
· Año.
· Precio.

Los vehículos se clasifican en autos compactos, autos de lujo, camionetas y vagonetas. Para los autos y vagonetas, también es importante almacenar la cantidad de pasajeros; mientras que para las camionetas se debe controlar la capacidad de carga en kgs. y la cantidad de ejes y de rodadas. Modele este sistema con diagramas de clases en UML e instancie cada una de las clases, asignándole datos mediante sus respectivas propiedades. Agregue un constructor a cada clase para inicializar sus datos e invoque el constructor de la clase base desde el constructor de cada clase derivada.

Ejercicio 5.6 Un equipo de futbol debe controlar las estadísticas de sus jugadores y para ello requiere un sistema computacional orientado a objetos con relaciones de herencia. Los datos de cada jugador son el número de uniforme, nombre y su posición (portero, defensa, medio y delantero). Además, el equipo desea almacenar la cantidad de goles anotados por cada jugador (excepto el portero) y los minutos jugados. Defina las clases con sus relaciones de herencia de la manera más apropiada para modelar este sistema e impleméntelas a través de un proyecto de formas como la siguiente:



Cuando el usuario seleccione un portero, entonces debe desactivarse el cuadro de texto de los goles anotados. Instancie cada una de las clases, capture sus datos para mostrarlos posteriormente. Agregue un constructor a cada clase para inicializar sus datos e invoque el constructor de la clase base desde el constructor de cada clase derivada.

Ejercicio 5.7 Codifica el siguiente código en C#, realiza las pruebas necesarias que permitan describir cual es el resultado que se obtiene. Diseñar también el diagrama de Clases mediante una herramienta UML.

```
public class Personal
{
    private int _Id;
    private string _Nombre;
    private string _Apellidos;

    public int Id
    {
        get { return _Id; }
        set { _Id = value; }
    }
    public string Nombre
    {
        get { return _Nombre; }
        set { _Nombre = value; }
    }
    public string Apellidos
    {
        get { return _Apellidos; }
        set { _Apellidos = value; }
    }

    public Personal(int pId, string pNombre, string pApellido)
    {
        Id = pId;
        Nombre = pNombre;
        Apellidos = pApellido;
    }
}
```

```
class Profesor : Personal
{
    public Profesor(int pId, string pNombre, string pApellido)
        : base(pId, pNombre, pApellido)
    {

    }
}
```

```
static void Main(string[] args)
{
    Profesor _Profesor = new Profesor(1, "Marcos", "Aguilar");
    Personal _Personal = new Personal(1, "julio", "Carcamo");

    Console.WriteLine("Clase Profesor");
    Console.WriteLine("Id : {0}", _Profesor.Id);
    Console.WriteLine("Nombre : {0}", _Profesor.Nombre);
    Console.WriteLine("Apellido : {0}", _Profesor.Apellidos);
    Console.WriteLine("-----------------------------------");
    Console.WriteLine("Clase Personal");
    Console.WriteLine("Id : {0}", _Personal.Id);
    Console.WriteLine("Nombre : {0}", _Personal.Nombre);
    Console.WriteLine("Apellido : {0}", _Personal.Apellidos);

    Console.ReadLine();
}
```

Ejercicio 5.8 Codifica el siguiente código en C#, realiza las pruebas necesarias que permitan describir cual es el resultado que se obtiene. Diseñar también el diagrama de Clases mediante una herramienta UML.

```
class Persona
{
 // Campo de cada objeto Persona que almacena su nombre
 public string Nombre;
 // Campo de cada objeto Persona que almacena su edad
 public int Edad;
 // Campo de cada objeto Persona que almacena su NIF
 public string NIF;

 // Incrementa en uno la edad del objeto Persona
 public virtual void Cumpleaños()
 {
   Console.WriteLine("Incrementada edad de persona");
 }

 // Constructor de Persona
 public Persona (string nombre, int edad, string nif)
 {
   Nombre = nombre;
   Edad = edad;
   NIF = nif;
 }
}

class Trabajador: Persona
{
 // Campo de cada objeto Trabajador que almacena cuánto gana
 int Sueldo;

 Trabajador(string nombre, int edad, string nif, int sueldo)
     : base(nombre, edad, nif)
 {// Inicializamos cada Trabajador en base al constructor de Persona
   Sueldo = sueldo;
 }

 public override Cumpleaños()
 {
   Edad++;
   Console.WriteLine("Incrementada edad de trabajador");
 }

 public static void Main()
 {
   Persona p = new Trabajador("Josan", 22, "77588260-Z", 100000);
   p.Cumpleaños();

 }
}
```

Unidad 6. Archivos

Competencias específicas a desarrollar:
• Utilizar técnicas de modelado para la solución de problemas • Aplicar la sintaxis de un lenguaje orientado a objetos. • Aplicar un Lenguaje orientado a objetos para la solución de problemas.
Material y equipo necesario:
• Equipo de cómputo con Lenguaje de Programación Microsoft Visual C# 2022 instalado. • Papel y lápiz

Introducción

Las variables y los arreglos ofrecen sólo un almacenamiento temporal de los datos; los datos se pierden cuando una variable local “queda fuera de alcance” o cuando el programa termina. En contraste, los archivos y las bases de datos se utilizan para una retención a largo plazo de grandes cantidades de datos, incluso después de que termine el programa que creó los datos. A menudo, los datos que se mantienen en archivos se conocen como datos persistentes.

Las computadoras almacenan archivos en dispositivos de almacenamiento secundario, como discos magnéticos, discos ópticos y cintas magnéticas. En esta última unidad, se presentan ejercicios donde el estudiante tendrá que crear, actualizar y procesar archivos de datos en programas con C#.

Ejercicio 6.1 Investiga en diversas fuentes de información acerca de los tipos de Excepciones que maneja C# y qué causa que se disparen estas excepciones, con el resultado de esta investigación crea una tabla donde se concentren los siguientes datos.

Tipo de Excepción	Causa que produce la Excepción

Ejercicio 6.2 Codifica el siguiente código en C#, realiza las pruebas necesarias que permitan describir cual es el resultado que se obtiene. Comentar el código para una mejor comprensión.

```
static void Main(string[] args)
{
    string fileName = "temp.txt";
    FileStream stream = new FileStream(fileName, FileMode.Open, FileAccess.Read);
    StreamReader reader = new StreamReader(stream);

    while (reader.Peek() > -1) Console.WriteLine(reader.ReadLine());
    reader.Close();
}
```

Ejercicio 6.3 Codifica el siguiente código en C#, realiza las pruebas necesarias que permitan describir cual es el resultado que se obtiene. Comentar el código para una mejor comprensión.

```
static void Main(string[] args)
{
    string fileName = "temp.txt";
    FileStream stream = new FileStream(fileName, FileMode.OpenOrCreate, FileAccess.Write);
    StreamWriter writer = new StreamWriter(stream);

    writer.WriteLine("Esta es la primera línea del archivo.");
    writer.Close();
}
```

Ejercicio 6.4 Codifica el siguiente código en C#, realiza las pruebas necesarias que permitan describir cual es el resultado que se obtiene. Comentar el código para una mejor comprensión.

```
static void Main(string[] args)
{
    string fileName = "temp.txt";
    StreamWriter writer = File.CreateText(fileName);

    writer.WriteLine("Este es mi Nuevo archivo creado.");
    writer.Close();
}
```

Ejercicio 6.5 Codifica el siguiente código en C#, realiza las pruebas necesarias que permitan describir cual es el resultado que se obtiene. Comentar el código para una mejor comprensión.

```
static void Main(string[] args)
{
    try
    {
        string fileName = "temp.txt";
        StreamWriter writer = File.AppendText(fileName);
        writer.WriteLine("Este es el texto adicionado.");
        writer.Close();
    }
    catch
    {
        Console.WriteLine("Error");
```

```
    }
}
```

Ejercicio 6.6 Codifica el siguiente código en C#, realiza las pruebas necesarias que permitan describir cual es el resultado que se obtiene. Comentar el código para una mejor comprensión.

```
static void Main(string[] args)
{
    try
    {
        string fileName = "temp.txt";
        int letter = 0;
        FileStream stream = new FileStream(fileName, FileMode.Open, FileAccess.Read);
        BinaryReader reader = new BinaryReader(stream);

        while (letter != -1)
        {
            letter = reader.Read();
            if (letter != -1) Console.Write((char)letter);
        }
        reader.Close();
        stream.Close();
    }
    catch
    {
        Console.WriteLine("Error");
    }
}
```

Ejercicio 6.7 Codifica el siguiente código en C#, realiza las pruebas necesarias que permitan describir cual es el resultado que se obtiene. Comentar el código para una mejor comprensión.

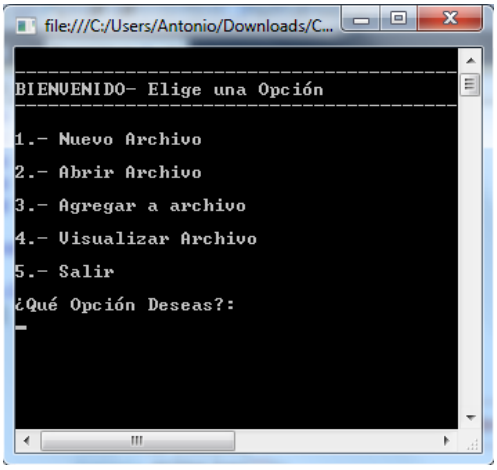
```
static void Main(string[] args)
{
    try
    {
        string fileName = "temp.txt";
        int[] data = {0, 1, 2, 3, 4, 5};
        FileStream stream = new FileStream(fileName, FileMode.Open, FileAccess.Write);
        BinaryWriter writer = new BinaryWriter(stream);

        for(int i=0; i<data.Length; i++)
        {
                writer.Write(data[i]);
        }

        writer.Close();
        stream.Close();
    }
    catch
    {
        Console.WriteLine("Error");
}
```

Ejercicio 6.8 Realizar un programa que contenga un menú de opciones en los cuales se puedan realizar operaciones sobre archivos, como; crear nuevo archivo,

abrir archivo, agregar información a un archivo y visualizar el contenido de un archivo.



Ejercicio 6.9 Codifica el siguiente código en C#, realiza las pruebas necesarias que permitan describir cual es el resultado que se obtiene. Particularmente analizar las excepciones, comentar el código para una mejor comprensión.

```
public static void Main()
  {
    StreamReader fichero;
    string nombre;
    string linea;

    Console.WriteLine("Introduzca el nombre del fichero");
    nombre = Console.ReadLine();

    try
    {
      fichero = File.OpenText(nombre);
      do
      {
        linea = fichero.ReadLine();
        if (linea != null)
        Console.WriteLine( linea );
      }
      while (linea != null);

      fichero.Close();
    }
    catch (Exception exp)
    {
      Console.WriteLine("Ha habido un error: {0}", exp.Message);
      return;
    }
  }
```

Ejercicio 6.10 Codifica el siguiente código en C#, realiza las pruebas necesarias que permitan describir cual es el resultado que se obtiene. Particularmente analizar las excepciones, comentar el código para una mejor comprensión.

```
public static void Main()
  {
    StreamWriter fichero;
    string nombre;
    string linea;
```

```
  Console.Write("Introduzca el nombre del fichero: ");
  nombre = Console.ReadLine();
  Console.Write("Introduzca la frase a guardar: ");
  linea = Console.ReadLine();

  try
  {
    fichero = File.CreateText(nombre);
    fichero.WriteLine( linea );
    fichero.Close();
  }
  catch (PathTooLongException e)
  {
    Console.WriteLine("Nombre demasiado largo!");
  }
  catch (IOException e)
  {
    Console.WriteLine("No se ha podido escribir!");
    Console.WriteLine("El error exacto es: {0}", e.Message);
  }
}
```

Glosario de Términos

Abstracción: Habilidad de tomar algo como un todo, sin tener en cuenta los detalles. El propósito de la abstracción es entender algo sin basarse en su estructura interna. La programación orientada a objetos se basa en la abstracción tanto de datos como funcional.

Argumento: Parte variable o entrada de una función.

Cadena: Secuencia de caracteres de texto (en inglés string).

Clase: Definición de un tipo de objetos que tienen unas características comunes. Una clase es un patrón para crear nuevos objetos. Una clase contiene tanto atributos como métodos.

Clase abstracta: Clase que se usa para definir las características comunes de un conjunto de clases. Una clase abstracta sirve como un patrón que define una serie de clases relacionadas. En C++ este término se usa específicamente para las clases que tienen un método virtual puro.

Clase base / padre / madre / ascendiente / superclase: Clase que define un conjunto de características comunes a todas las clases derivadas de ella.

Clase derivada / hija / descendiente / subclase: Clase que es descendiente o es derivada de otra. Hereda las propiedades de la clase base.

Clases genéricas o parametrizadas (templates): Plantilla para generar clases reales que puedan diferir solamente en unos parámetros y no en el código fuente. Los parámetros pueden ser tipos o clases y otros valores constantes, como por ejemplo el tamaño de una pila.

Constructor: En C++ una operación que inicializa el objeto al crearse.

Declaración: Una declaración introduce nombres y sus tipos dentro del programa sin que necesariamente se defina un objeto o función asociado. Sin embargo, muchas declaraciones sirven como definiciones.

Destructor: En C++ es la operación que destruye un objeto eliminando todos sus componentes y realizando tareas de limpieza o de conclusión de operaciones.

Diagrama de clases (UML): Diagrama que representa un conjunto de clases, interfaces y sus relaciones.

Diseño: Proceso de convertir los requisitos de un sistema en una manera de resolver el problema con el objetivo de posibilitar una implementación que cumpla el coste, prestaciones y calidad deseados.

Encapsulación: Habilidad de poner todos los datos y funcionalidades de un tipo de datos en una estructura controlada y abstracta, de forma que esté aislada del resto del sistema. Las clases permiten al programador la libertad de encapsular atributos y métodos en una única estructura.

Escenario (UML): Secuencia específica de acciones que ilustra un comportamiento.

Espacio de nombres: Ámbito en el que se pueden definir nombres asociados a un identificador.

Especialización: Creación de clases derivadas de una clase base refinando ésta. Consiste en añadir o modificar métodos.

Excepción: Es una situación anómala o error que se puede presentar durante la ejecución de un programa. Elevar una excepción es abandonar la ejecución normal de un programa para indicar que el error ha tenido lugar. El manejador de excepciones se encargará de su tratamiento.

Generalización: Relación inversa a la de especialización, en la cual se identifican las características comunes a todos los objetos que se generalizan y se ignoran las diferencias.

Herencia múltiple: Habilidad de heredar de varias clases bases.

Instancia: Realización de una clase. Un objeto es una instancia de una clase.

Instanciación: Acción de instanciar una clase, es decir, el proceso de crear un objeto. Similar a construcción.

Interacción: Comportamiento que comprende un conjunto de mensajes que se intercambian entre un conjunto de objetos.

Iterador: Mecanismo que permite recorrer una estructura que contiene muchos objetos como, por ejemplo, un contenedor.

Jerarquía de clases: Cuando unas clases heredan de otras se forma una estructura arbórea que se denomina jerarquía. Se puede pensar una jerarquía de clases como un árbol de familia, en el que las clases derivadas son las hijas de las clases base (madres).

Lenguaje orientado a objetos: Lenguaje que tiene la capacidad de definir clases y objetos y que permite la herencia y el polimorfismo.

Objeto: Es la instancia de una clase. Un objeto es una entidad que tiene un estado y un conjunto definido de operaciones (métodos) que operan sobre este estado. El estado se representa por un conjunto de atributos del objeto. Las operaciones asociadas con el objeto dan servicio a otros objetos (clientes) que piden estos servicios cuando se necesita alguna operación (por medio de mensajes).

Operador: Función que tiene un nombre, generalmente no alfanumérico, y que se invoca sin paréntesis y en distintas posiciones (prefijo, postfijo o infijo).

Parámetro: Parte variable de una plantilla o macro que permite obtener instancias diferentes.

Polimorfismo: Habilidad de enviar el mismo mensaje a objetos de diferentes clases. Estos objetos reciben el mismo mensaje pero responderán de distinta forma. El polimorfismo en C++ se consigue a través de los métodos virtuales y punteros o referencias a clases bases.

Programación orientada a objetos (POO): Estilo de programación que conduce a un sistema software basado en la creación y manipulación de objetos.

Protegido (referido a un miembro de una clase): Miembro sólo accesible por métodos de su clase y descendientes.

Público (referido a un miembro de una clase): Miembro accesible por métodos de cualquier clase.

Sobrecarga: Es la propiedad de que un mismo identificador tenga varios significados. En C++, C# hay sobrecarga en las funciones y los operadores, donde puede haber un mismo nombre de función para distintas operaciones sobre distintos argumentos. La llamada a una función u operador sobrecargado se resuelve en tiempo de compilación basándose en el tipo de argumentos con el que se llama.

Stream: Estructura de datos que permite el flujo, generalmente secuencial, de objetos a través de él. Utilizado para la E/S en C++ y C#.

Subclase: Ver clase derivada.

Superclase: Ver clase base.

This: En C++ designa un puntero al objeto sobre el que opera un método.

Variable: Entidad que puede cambiar de estado. En C++ una variable es un dato de un tipo fundamental o derivado (por ejemplo int, float, punteros), en contraposición a los objetos que son instancias de una clase.

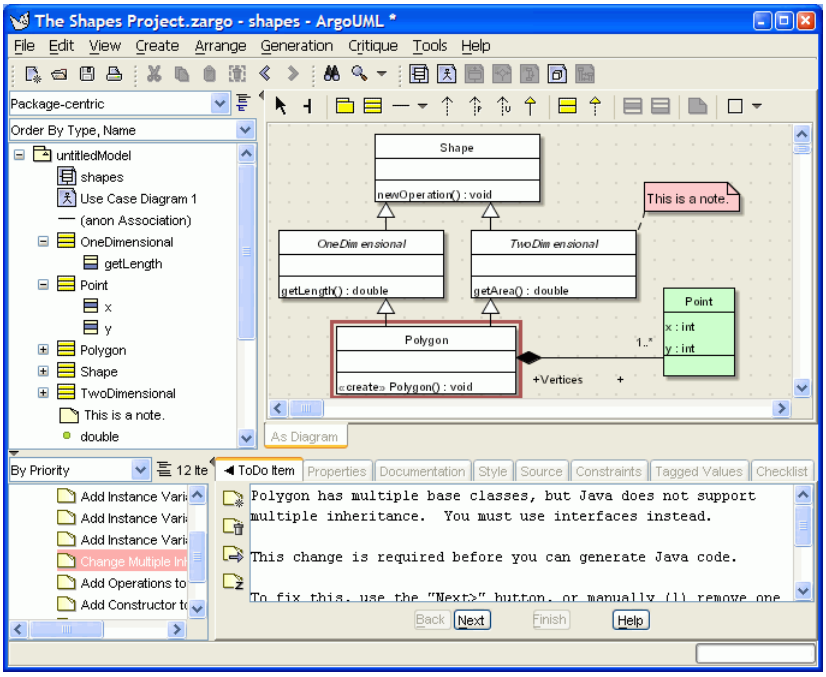
Anexos

I. Herramienta CASE para el Modelado de Diagramas de Clases de UML

ArgoUML es una aplicación de diagramado de UML escrita en Java y publicada bajo la Licencia BSD. Dado que es una aplicación Java, está disponible en cualquier plataforma soportada por Java.

Mmaneja los siguientes tipos de diagrama:

- Clases
- Estados
- Casos de Uso
- Actividad
- Colaboración
- Desarrollo
- Secuencia



Enlace de descarga: http://argouml.softonic.com/descargar

II. Tabla con la Jerarquía de Operadores manejados en la mayoría de los Lenguajes de Programación.

Prioridad	Operador	Descripción
1º	( )	Signo de agrupación
2º	-	Negativo (-1)
3º	%	Porcentaje
4º	^	Potenciación
5º	* /	Multiplicación y división
6º	+ -	Suma y resta
7º	&	Concatenación
8º	= < > <= >= <>	Comparación

III. Tabla de Código ASCII

33 !	54 6	75 K	96 `	117 u	138 Š	159 Ÿ	180 ´	201 É	222 Þ	243 ó
34 "	55 7	76 L	97 a	118 v	139 ‹	160	181 µ	202 Ê	223 ß	244 ô
35 #	56 8	77 M	98 b	119 w	140 Œ	161 ¡	182 ¶	203 Ë	224 à	245 õ
36 $	57 9	78 N	99 c	120 x	141 □	162 ¢	183 ·	204 Ì	225 á	246 ö
37 %	58 :	79 O	100 d	121 y	142 □	163 £	184 ¸	205 Í	226 â	247 ÷
38 &	59 ;	80 P	101 e	122 z	143 □	164 ¤	185 ¹	206 Î	227 ã	248 ø
39 '	60 <	81 Q	102 f	123 {	144 □	165 ¥	186 º	207 Ï	228 ä	249 ù
40 (	61 =	82 R	103 g	124 |	145 ‘	166 ¦	187 »	208 Ð	229 å	250 ú
41)	62 >	83 S	104 h	125 }	146 ’	167 §	188 ¼	209 Ñ	230 æ	251 û
42 *	63 ?	84 T	105 i	126 ~	147 “	168 ¨	189 ½	210 Ò	231 ç	252 ü
43 +	64 @	85 U	106 j	127 □	148 ”	169 ©	190 ¾	211 Ó	232 è	253 ý
44 ,	65 A	86 V	107 k	128 €	149 •	170 ª	191 ¿	212 Ô	233 é	254 þ
45 -	66 B	87 W	108 l	129 □	150 –	171 «	192 À	213 Õ	234 ê	255 ÿ
46 .	67 C	88 X	109 m	130 ‚	151 —	172 ¬	193 Á	214 Ö	235 ë	256
47 /	68 D	89 Y	110 n	131 ƒ	152 ˜	173 -	194 Â	215 ×	236 ì	
48 0	69 E	90 Z	111 o	132 „	153 ™	174 ®	195 Ã	216 Ø	237 í	
49 1	70 F	91 [	112 p	133 …	154 š	175 ¯	196 Ä	217 Ù	238 î	
50 2	71 G	92 \	113 q	134 †	155 ›	176 °	197 Å	218 Ú	239 ï	
51 3	72 H	93]	114 r	135 ‡	156 œ	177 ±	198 Æ	219 Û	240 ð	
52 4	73 I	94 ^	115 s	136 ˆ	157 □	178 ²	199 Ç	220 Ü	241 ñ	
53 5	74 J	95 _	116 t	137 ‰	158 □	179 ³	200 È	221 Ý	242 ò	

Bibliografía

1. Deitel, Harvey M. y Paul J. Deitel. (2007). Cómo programar en C#. México: Pearson Educación.
2. Joyanes Aguilar, Luis. Java 2: Manual de Programación. 4ª edición. Ed. Prentice Hall.
3. 2. Joyanes Aguilar, Luis. Programación en Java 2 Algoritmos, Estructura de Datos y Programación Orientada a Objetos. Ed. Prentice Hall.

Linkografía

1. http://msdn.microsoft.com/es-es/library/
2. http://www.itnuevolaredo.edu.mx/maestros/sis_com/takeyas/Apuntes/POO/index.htm
3. http://codigofuentenet.wordpress.com/
4. http://www.mundoprogramacion.com/colabora/NET2005/svalsse_leyendo_y_escribiendo_archivos.htm
5. http://argouml.softonic.com/descargar